Day 2 Assignment

JVM Tuning and Spring Boot Microservice Performance

John Michael Bilbao
Techstart
August 10, 2026

Introduction

A microservice is just a small application that does one job and talks over HTTP. I used Spring Boot for this one because it comes with Tomcat built in and handles most of the setup itself, so the whole service is only a few lines of code.

Spring Boot runs on the JVM, and the JVM decides how much memory the application gets and when it clears out objects that are no longer needed. That clearing out is garbage collection. If the heap is too small the collector keeps running, and the application spends more time freeing memory than answering requests. Tuning means setting options like the heap size and the collector type to fit the work the application actually does. I built a small item service, put it under load, watched it in VisualVM, then changed the settings to see what would happen. Everything here ran on Java 21 with Spring Boot 3.4.1, on a machine with eight cores.

The microservice

It is a Maven project with the Spring Web and Actuator dependencies. Three classes, and the controller is the only interesting one:

```
@RestController
@RequestMapping("/api/items")
public class ItemController {

   private static final String[] CATEGORIES =
         {"Tools", "Books", "Food", "Toys", "Parts"};

   @GetMapping
   public List<Item> getItems(@RequestParam(defaultValue = "5000") int count) {
      List<Item> items = new ArrayList<>();
      for (int i = 0; i < count; i++) {
         String name = "Item " + i + " - " + CATEGORIES[i % CATEGORIES.length];
         items.add(new Item(i, name,
               CATEGORIES[i % CATEGORIES.length], 10.0 + (i % 100)));
      }
      return items;
   }
}
```

Item is a record with an id, name, category and price. The endpoint is GET /api/items?count=5000 and it returns the list as JSON.

The list gets rebuilt on every request instead of being cached, which is on purpose. A cached list would barely allocate anything and there would be nothing to watch in VisualVM.

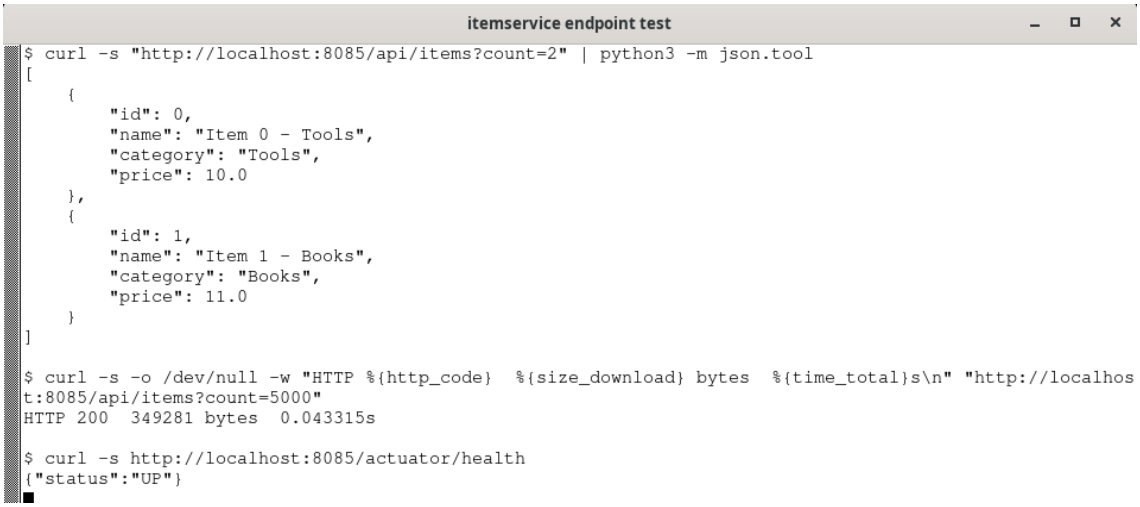

Figure 1: The endpoint responding, and the health check

Monitoring with VisualVM

I got VisualVM from visualvm.github.io and started it while the service was already running. The process shows up on the left under Local, and double clicking it opens the Overview tab, which shows the JVM options the process started with. I ran the first round with a small heap and the serial collector so there would be something to find:

```
java -Xms64m -Xmx128m -XX:+UseSerialGC \
    -cp "target/classes:$(cat cp.txt)" \
    com.example.itemservice.ItemserviceApplication
```

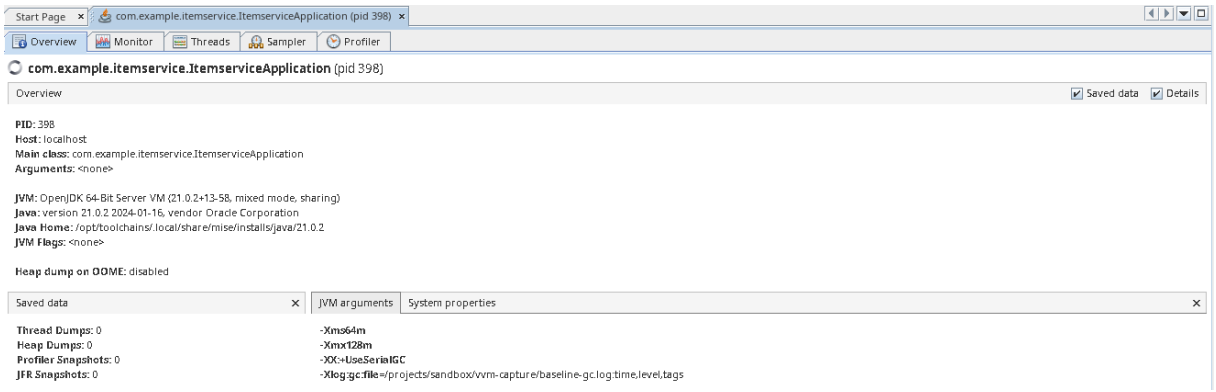

Figure 2: Overview tab, showing the baseline flags in effect

Nothing much happens while the application is idle, so I wrote a Python script that hits the endpoint from several threads at once and left it looping for a couple of minutes while I watched the Monitor tab.

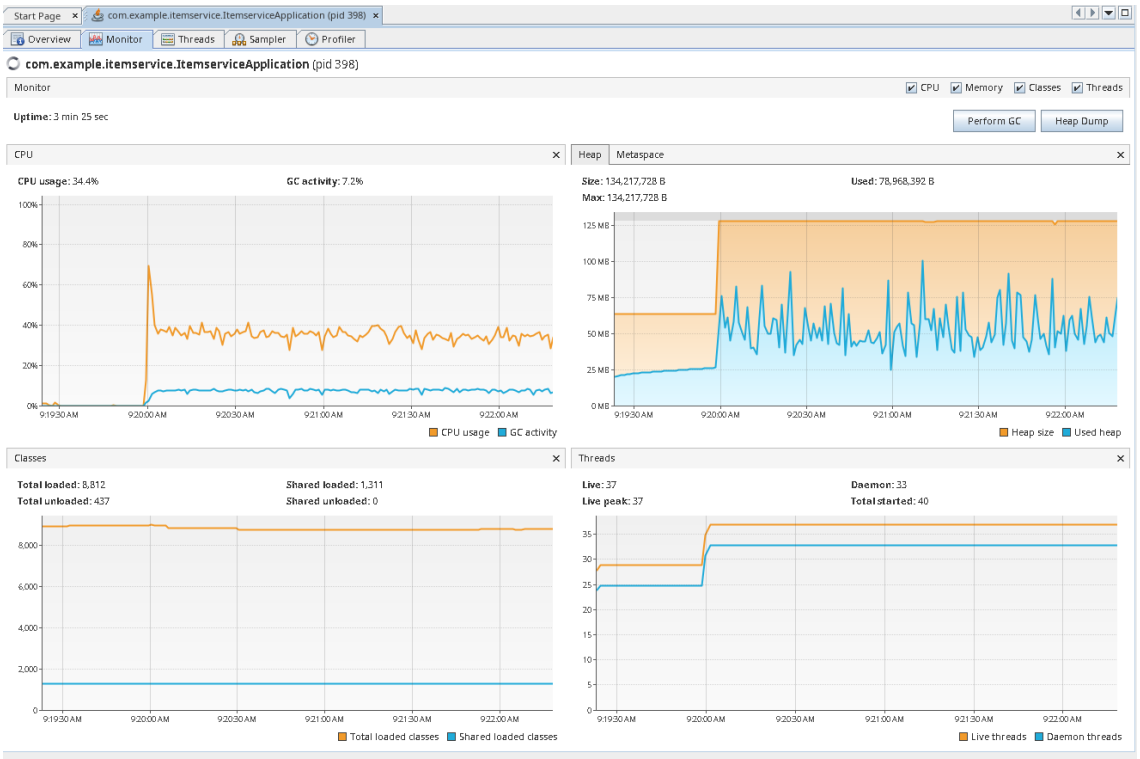

Figure 3: Monitor tab during the baseline run

Heap went straight up to its ceiling and stayed pinned there, and used heap sawtoothed up and down without ever settling, so the collector was running constantly rather than every now and then. Threads went up when the load arrived and then stayed flat, so the thread count was not the problem.

The GC activity figure confused me at first. VisualVM showed about 7 percent, which sounds harmless, but the GC log for the same session was full of collections and showed the application frozen for most of the run. Both are right. Serial GC stops everything and then works on a single thread, and VisualVM spreads GC time across all the cores, so one busy core out of eight looks small. The graph shows how much of the CPU is going into GC, while the log shows how long the application is actually stopped, and for a service answering requests it is the second one that matters.

Changing the settings

The heap was the problem, not CPU and not threads. I raised the maximum from 128 MB to 512 MB and set the minimum to match, which stops the JVM growing the heap while it is already busy. I also swapped the serial collector for G1, since G1 spreads its work across cores and does much of it concurrently, and gave it a 100 ms pause target.

```
java -Xms512m -Xmx512m -XX:+UseG1GC -XX:MaxGCPauseMillis=100 \
    -cp "target/classes:$(cat cp.txt)" \
    com.example.itemservice.ItemserviceApplication
```

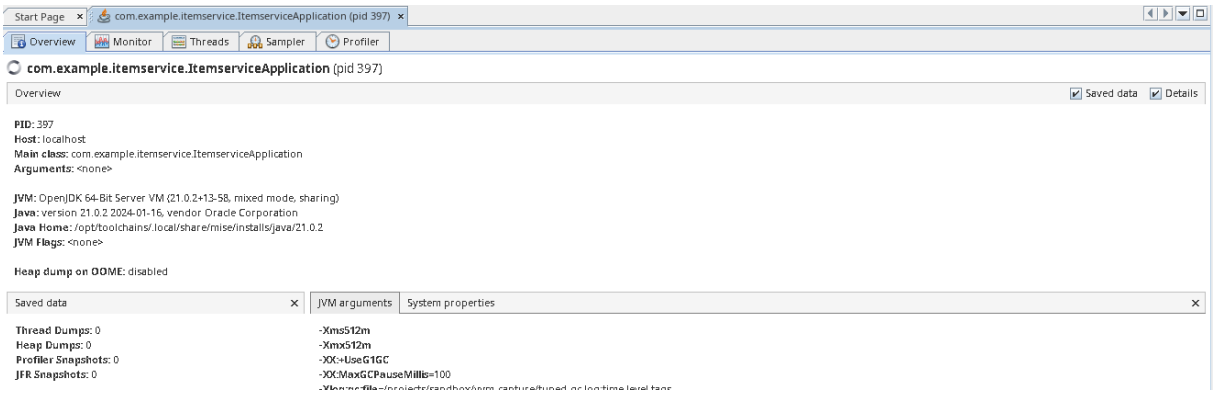

Figure 4: Overview tab after the change

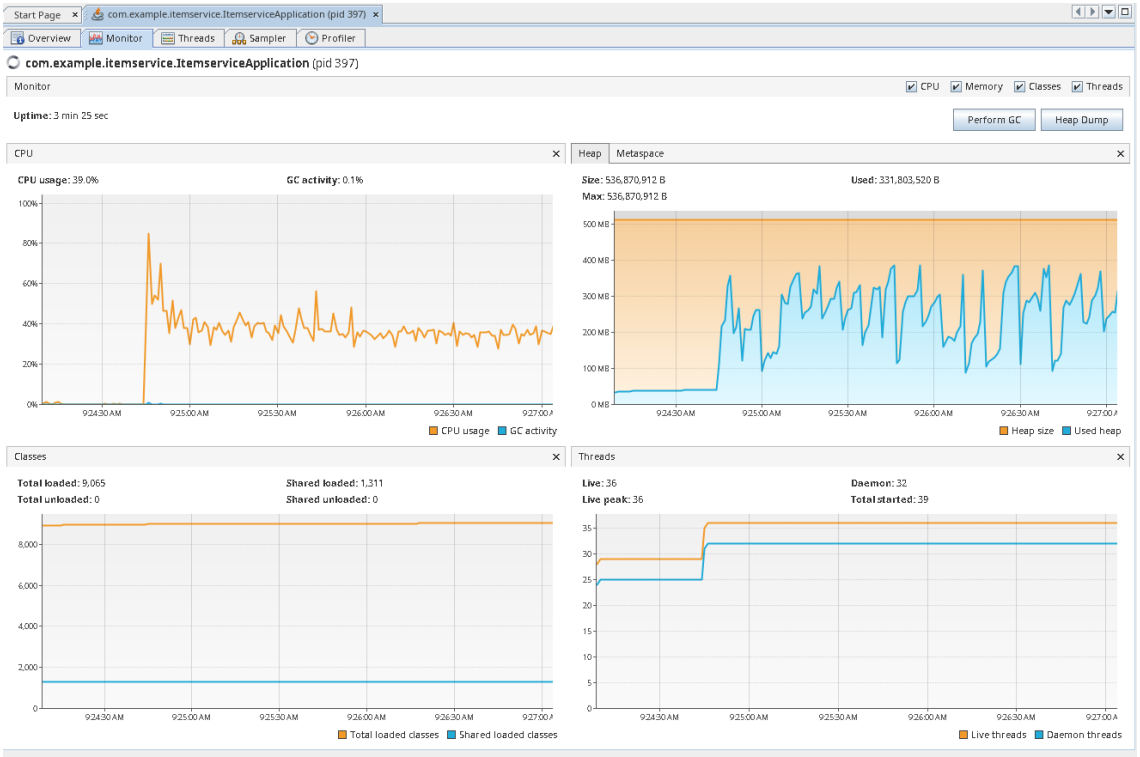

Figure 5: Monitor tab during the tuned run

Used heap now moves up and down with visible gaps instead of one solid band, so the collector gets to rest between cycles. GC activity dropped to almost nothing and the full collections disappeared completely. CPU actually went up a little, which looked wrong until I realised it was finally being spent on requests instead of on collecting garbage.

Results

Screenshots show behaviour but not speed, so I ran the load test again with VisualVM detached. Same code and same machine, only the flags were different.

	Before	After
Requests per second	103	174
Average response time	155 ms	92 ms
Slowest 5% of requests	288 ms	115 ms
Full collections	76	0
Total time paused	9.8 s	0.2 s

The full collections are what stand out. Each one freezes every thread while it works, and that is what made the slowest requests slow.

I had run a lighter test earlier and it barely moved at all. The GC work still dropped, but the application was not waiting on garbage collection at that load in the first place, so there was nothing for the change to fix. The improvement only turned up once the load was heavy enough to run the small heap out of room.

What I learned

Tuning only helps if the thing you are tuning is what is actually slowing you down. The lighter test made that obvious, since the GC work dropped a long way and the response times did not move at all.

It is also worth checking what a percentage is a percentage of. The GC activity on screen looked fine sitting next to a log that showed the application frozen for most of the run, and neither number was wrong.

Repeated full collections turned out to be a better warning sign than a heap sitting near its limit, since a full heap can just mean the collector is doing its job well. And bigger is not automatically better. I went with 512 MB because that was what got rid of them. Java's own default on this machine was a much larger heap with G1 already chosen, and the endpoint never struggled, so I had to shrink it on purpose to create a problem worth looking at.